SMU Lyle School of Engineering

Reinforcement Learning

Markov Decision Processes (MDP)

Sutton/Barto* Chapter 3

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018

The Agent–Environment Interface

- MDPs are meant to be a straightforward framing of the problem of learning from interaction to achieve a goal.
- Actions influence the future rewards by leading to different states.

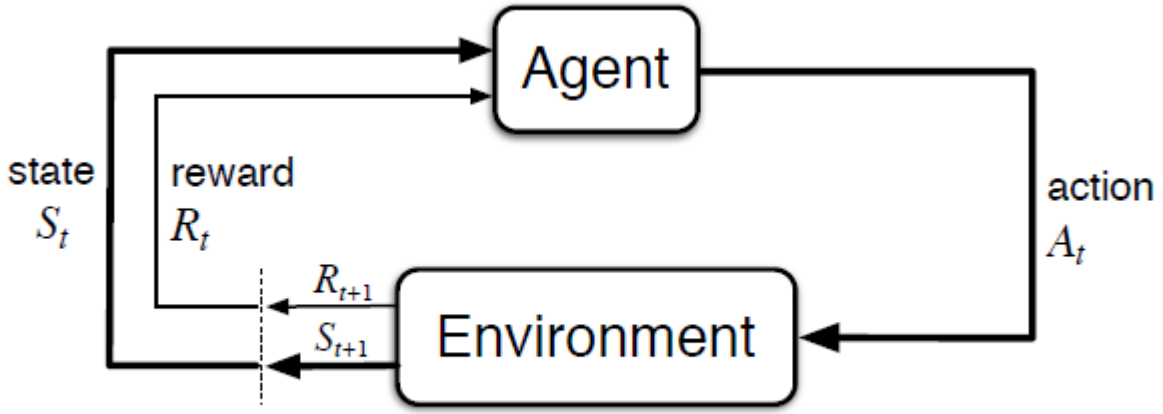


Figure 3.1: The agent–environment interaction in a Markov decision process.

Interaction is a sequence of discrete time steps, $t = 0, 1, 2, 3, \ldots$

- At each time step t:
  1. Agent receives some representation of the environment's state $S_t \in \mathcal{S}$
  2. Agent selects an action, $A_t \in \mathcal{A}(S_t)$
  3. One time step later (as a consequence of its action) the agent receives a numerical reward, $R_{t+1} \in \mathcal{R}$ and finds itself in a new state S_{t+1}

Leads to sequence or trajectory that begins like this:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$$

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

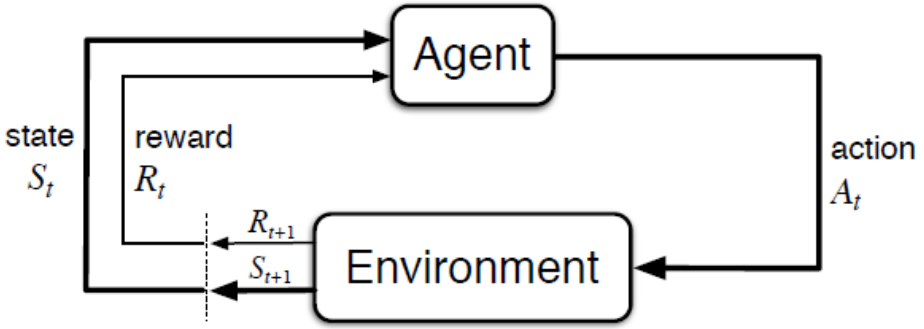
MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a
	expected immediate reward from state s after action a
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or v_*
$\bar{V}_t(s)$	expected approximate action value
U_t	target for estimate at time t

Finite MDP



- An MDP is **finite** if the
  - sets of states ($\mathcal{S}$),
  - actions ($\mathcal{A}$), and
  - rewards ($\mathcal{R}$) all have a finite number of elements.

- **Dynamics** of the MDP:

$$p(s', r|s, a) \stackrel{\text{def}}{=} \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}$$

- As a deterministic functions:

$$p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0,1]$$

- A decision process is a MDP if p completely characterizes the environment's dynamics.
- Dynamics only depends on the previous state s, so it must have sufficient information about the past. If this is true then it is called an **information state** and the system has the **Markov property**.

Transition and Reward Functions

- State transition probabilities:

$$p(s'|s,a) \stackrel{\text{def}}{=} \Pr\{S_t = s', \mid S_{t-1} = s, A_{t-1} = a\}$$

$$= \sum_{r \in \mathcal{R}} p(s', r|s, a)$$

- Expected reward for state-action pairs:

$$r(s,a) \stackrel{\text{def}}{=} \mathbb{E}[R_t| \, S_{t-1} = s, A_{t-1} = a]$$

$$= \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a)$$

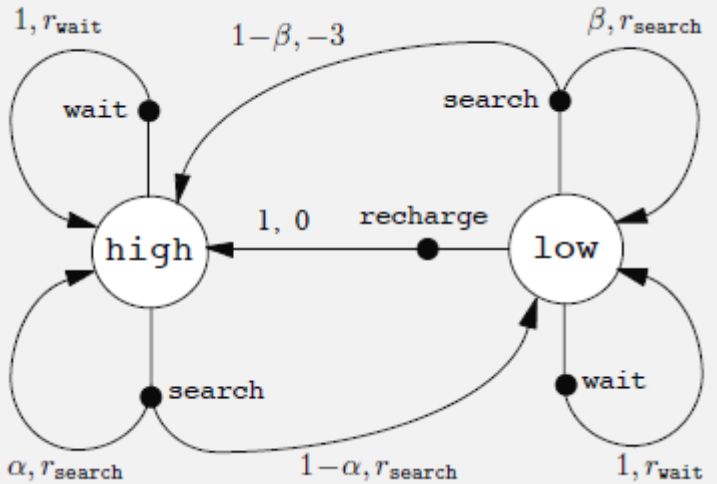
- Expected rewards for state-action-next state triples:

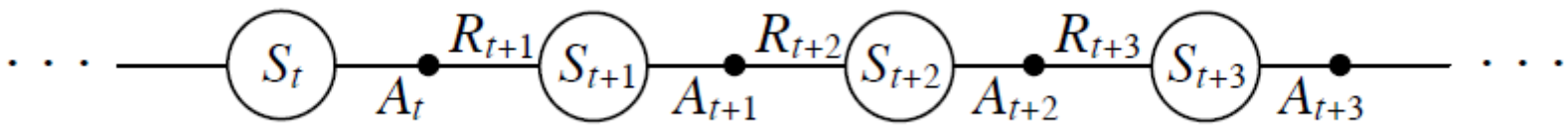
$$r(s,a,s') \stackrel{\text{def}}{=} \mathbb{E}[R_t| \, S_{t-1} = s, A_{t-1} = a, S_t = s']$$

$$= \sum_{r \in \mathcal{R}} r \frac{p(s', r|s, a)}{p(s'|s, a)}$$

Add an Example!!!!!!!!!!!!

s	a	s'	$p(s' \mid s, a)$	$r(s, a, s')$
high	search	high	α	$r_{\texttt{search}}$
high	search	low	$1-\alpha$	$r_{\texttt{search}}$
low	search	high	$1-\beta$	-3
low	search	low	β	$r_{\texttt{search}}$
high	wait	high	1	$r_{\texttt{wait}}$
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	$r_{\texttt{wait}}$
low	recharge	high	1	0
low	recharge	low	0	-





Goals and Rewards

- Goal: maximize the expected value of the cumulative sum of the received reward signal.

- The reward signal at time t is a single number:

$$R_t \in \mathcal{R} \in \mathbb{R}$$

- Note:
  - Reward signals should communicate what the agent wants to achieve and not how to do it!
  - Typically, only provide reward for the final goal.

Return and Episodes

- Goal is to maximize the expected return

$$G_t \stackrel{\text{def}}{=} R_{t+1} + R_{t+2} + R_{t+3} + \cdots R_T$$

  where T is the final step.

- Episodic task:
  - Episodes end with a terminal state at time T.
  - Typically followed by a reset for the next episode.

- Continuing task:
  - Has no episodes! $T = \infty$

Reward Discounting

- Discounting is often used when rewards now are more important then rewards later. Continuing tasks always use discounting.

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

  where $0 \leq \gamma \leq 1$ is the discount factor.

- Properties:
  - Discounting guarantees a finite sum for G_t if $\{R_k\}$ is bounded for continuing tasks
  - "myopic" agents use $\gamma = 0$ and only maximizes immediate rewards.
  - Larger γ make the agent more farsighted.

- Return is recursive:

$$\begin{aligned} G_t &\stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \\ &= R_{t+1}\, \gamma\, (R_{t+2} + R_{t+3} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

Policy

(Stochastic) **Policy**: a mapping from states to probabilities of selecting each possible action.

$$\pi(a|s)$$

- Following policy π at time t means to chose action a in state s with probability $\pi(a|s)$

Deterministic policy: For each state prescribes a single action.

$$a = \pi(s)$$

- This just means we have a stochastic policy that has a probability of 1 for exactly one action.
- Finite MDP have an optimal deterministic policy.

Value Functions

- Most RL algorithms estimate a value function for a policy.
- State value function: Expected value of being in a state.

$$v_\pi(s) \overset{\text{def}}{=} \mathbb{E}_\pi[G_t \mid S_t = s]$$

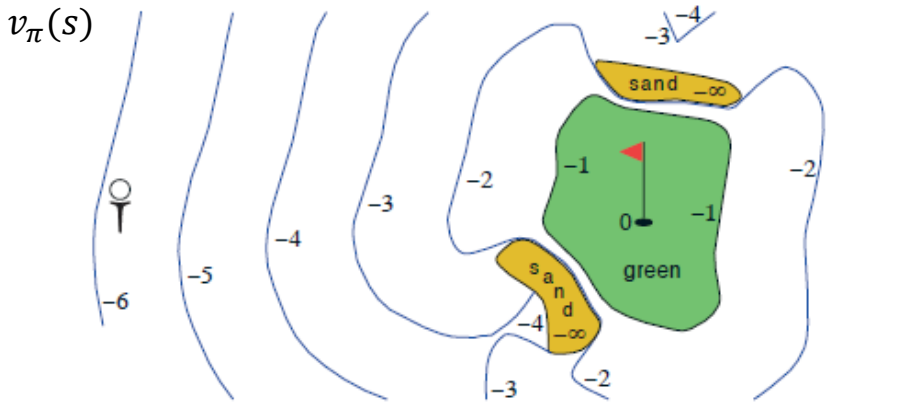
$$= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s\right] \quad \text{for all } s \in \mathcal{S}$$

- Action value function: Expected value of choosing an action in a state.

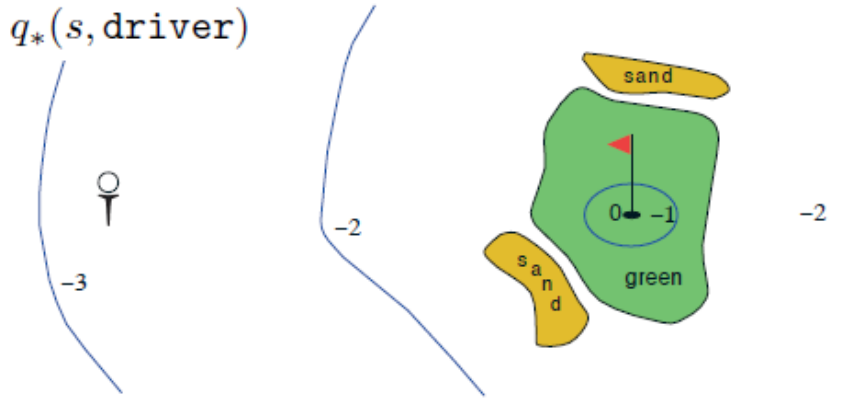
$$q_\pi(s,a) \overset{\text{def}}{=} \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

$$= \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a\right] \quad \text{for all } s \in \mathcal{S}$$

Each stroke: $r = -1$



State-value function for policy π that always uses the putter.



Optimal action-value function for using the driver.

$\mathbb{E}_\pi$ … Expectation under policy π = follow the policy

Value Function Estimation and Monte Carlo Methods

- The value functions v_π and q_π can be estimated from experience.

- Example:
  - agent follows policy π.
  - maintains an average $\bar{V}(s)$ of the actual returns that have followed that state. The average is updated at each visit.
  - $\lim_{visits \to \infty} \bar{V}(s) = v_\pi(s)$
- This sample-average method is a simple Monte Carlo method.

- This also works for $q_\pi(\mathrm{s}, \mathrm{a})$

- For problems with many states: learn a function with a smaller number of parameters.

Bellman Equation

Value functions always have a **recursive relationship** between the

- value of a state, and the
- value of its successor.

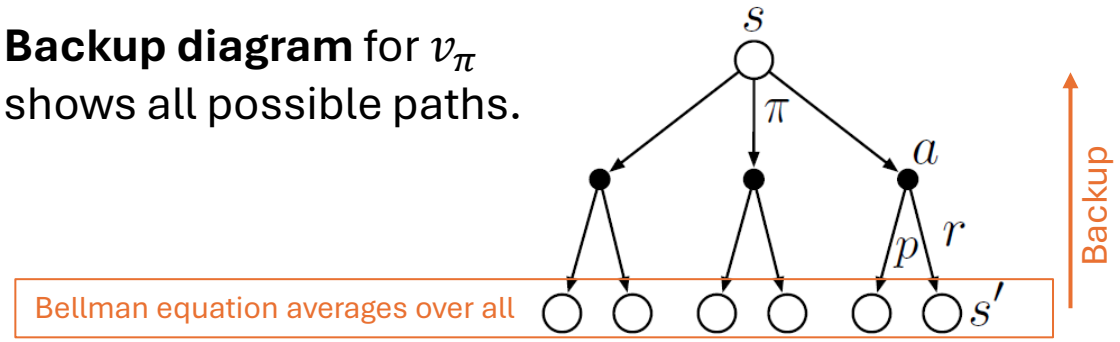
Value of a state = reward + value of successor

$$
\begin{aligned}
v_\pi(s) &\overset{\text{def}}{=} \mathbb{E}_\pi[G_t | S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \\
&= \underbrace{\sum_a \pi(a|s)} \sum_{s',r} p(s', r|s, a)[r + \gamma v_\pi(s')] \quad \text{for all } s \in \mathcal{S}
\end{aligned}
$$

Expectation

There are $|\mathcal{S}|$ Bellman equations.

Backup diagram for v_π shows all possible paths.



Optimal Policies and Value Functions

- Value functions define a partial ordering over policies.
- A policy is better if it has larger state values.

$$\pi \geq \pi' \Leftrightarrow v_\pi(s) \geq v_{\pi'}(s) \text{ for all } s \in \mathcal{S}$$

- There is always an **optimal policy** π_* with the optimal state-value function v_* defined as

$$v_*(s) \stackrel{\text{def}}{=} \max_\pi v_\pi(s) \text{ for all } s \in \mathcal{S}$$

- Implies an optimal action-value function:

$$q_*(s,a) \stackrel{\text{def}}{=} \max_\pi q_\pi(s,a) \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}$$

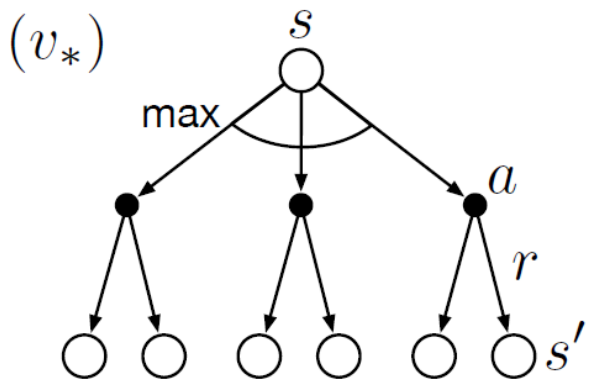
- Since

$$q_*(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

Bellman Optimality Equation

- Idea: the value of a state under an optimal policy must equal the expected return for the best action from that state.

$$
\begin{aligned}
v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s,a) \\
&= \max_a \mathbb{E}_\pi \left[G_t | S_t = s, A_t = a \right] \\
&= \max_a \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a \right] \\
&= \max_a \mathbb{E}_\pi \left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a \right] \\
&= \max_a \sum_{s',r} p(s', r \,|s, a)[r + \gamma v_*(s')]
\end{aligned}
$$



Bellman Optimality

- The optimality condition is a system of $|\mathcal{S}|$ non-linear equations of the from:

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a)$$

- Guarantee: Finite MDPs have a unique solution.

- Issues of solving the system:
  - The Markov property for states has to be satisfied.
  - p needs to be completely known.
  - Computational resources for solving the system with many states.

- This is rarely possible in real applications and we need approximations.

Summary

Terms

- **Actions** are the agent's choices
- **States** are the basis for the choices
- **Reward** is used to evaluate choices
- **Objective**: maximize reward over time
- **Policy**: a stochastic rule to select actions as a function of the state

Finite MDP

- **Complete knowledge**: solve the MDP directly to optimality. Only feasible for small problems
- **Incomplete knowledge**: RL focuses on this case by using approximation.

Tasks

- **Policy evaluation** (prediction): estimate v_π or q_π for a given fixed π.
- **Control**/policy improvement: find π_*